NYC Parking Violations - Implementação

UNIDADE CURRICULAR DE BIG DATA

Introdução

Nesta segunda parte do trabalho é possível explorar conceitos lecionado em aula como por exemplo:

- Arquitetura;
- Controlo de Acessos;
- MapReduce & Aggregations;
- Visualização de Dados & Valor;

Arquitetura

Distribuição dos dados por 5 ***Shards***, 1 para cada ano:

2020

2021

2022

2023

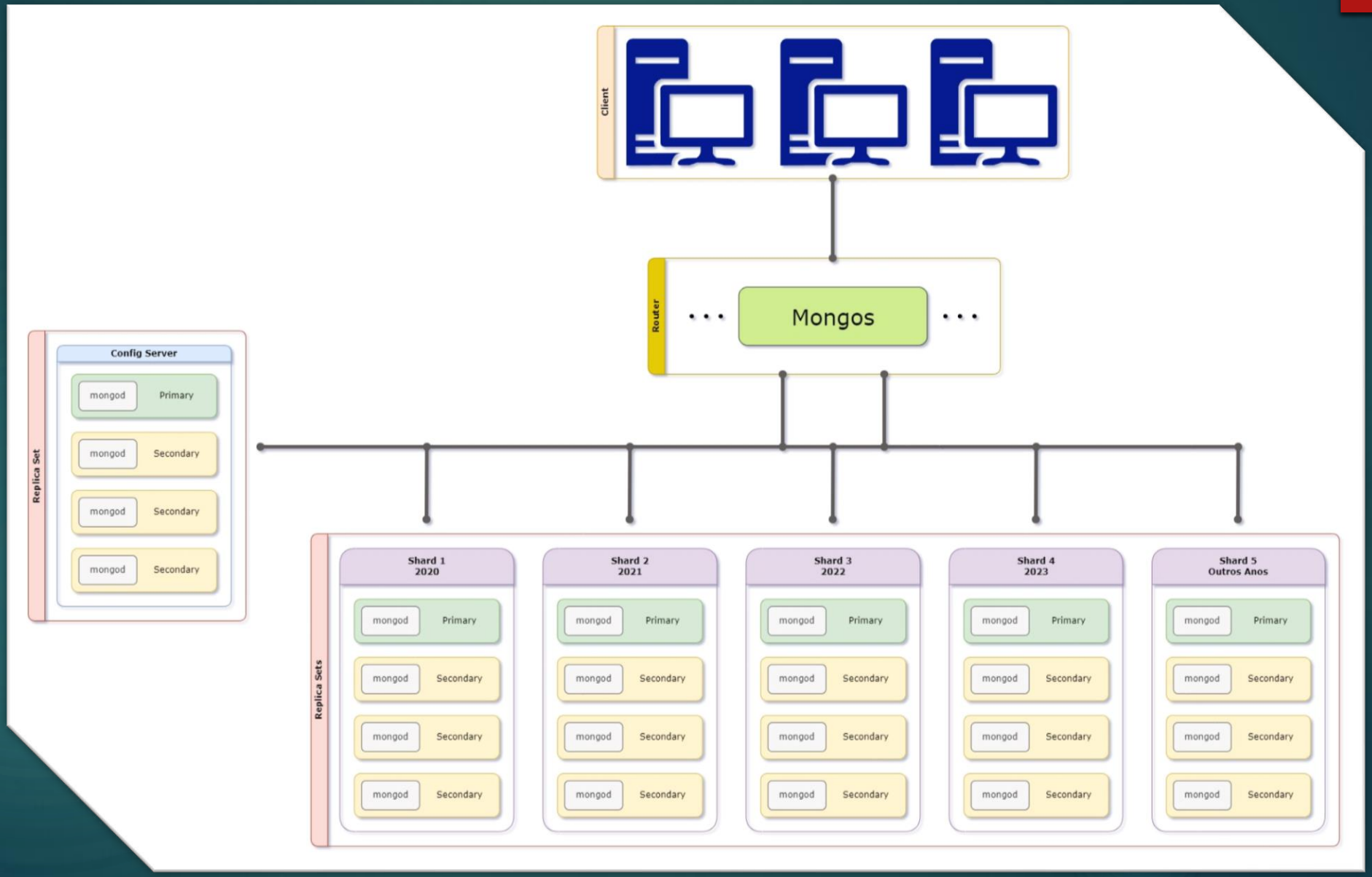
Issue Date Errado

Arquitetura – Adição do Issue Year

- Realizou-se um pré-processamento dos dados para criar uma nova coleção.
- Para cada documento existe um novo campo denominado de ***Issue Year***, o comando a seguir realiza a sua criação.

```
db.NYC_Parking_Violations_Collection.aggregate(
    [
        {
        $addFields: {
            "Issue Year": { $substr: [ "$Issue Date", 6, 4 ] }
        }
    },
    {
        $out: "NYC_Parking_Violations_Collection_Updated"
    }
    ]
)
```

Diagrama da Arquitetura



Arquitetura

Router – Roteia *queries* e operações de escrita em ***Shards*** num ***Sharded Cluster***

Config Server – Guarda metadados de um ***Shard Cluster***

Hashing & Sharding – Utilizamos o modelo *range* da seguinte maneira

Shard 1 – *Issue Year* para executar tarefa de ***Hashing*** e ter todos os documentos do ano 2020;

Shard 2 – *Issue Year* para executar tarefa de ***Hashing*** e ter todos os documentos do ano 2021;

Shard 3 – *Issue Year* para executar tarefa de ***Hashing*** e ter todos os documentos do ano 2022;

Shard 4 – *Issue Year* para executar tarefa de ***Hashing*** e ter todos os documentos do ano 2023;

Shard 5 - *Issue Year* para executar tarefa de ***Hashing*** e ter todos os documentos com valores diferentes dos representados nos shards acima;

Todos os ***Shards*** utilizariam o modelo distribuído ***Replica Set***;

Controlo de Acessos - Admin

- Foi implementado o controlo de acessos, ***Role-Based Access Control***, onde é possível criar utilizadores e regras de acesso.
- Para exemplificar existem 3 modelos de utilizadores: ***Admin***, ***Read-Write***, ***Read-Only***.

```
db.createUser(
    {
        user: "Admin",
        pwd: "*******",
        roles: [
            {
                role: "dbOwner",
                db: "NYC_Parking_Violations"
            }
        ]
    }
)
```

Controlo de Acessos - Read-Write

É possível visualizar o ***Read-Write***

```
db.createUser(
    {
        user: "Read-Write",
        pwd: "*******",
        roles: [
            {
            role: "readWrite",
            db: "NYC_Parking_Violations"
            }
        ]
    }
)
```

Controlo de Acessos - Read-Only

- É possível visualizar o ***Read-Only***

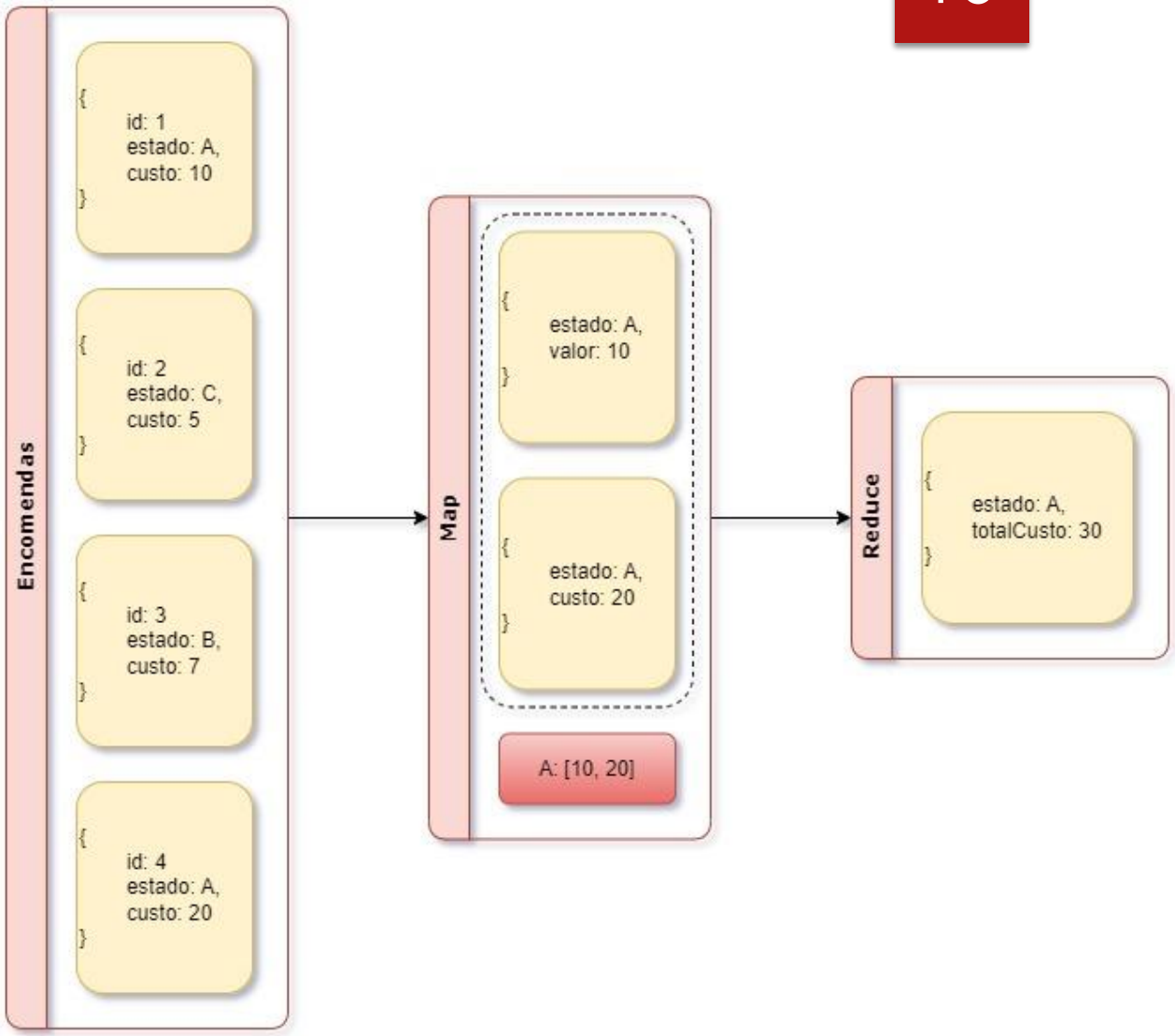
```
db.createUser(
    {
        user: "Read-Only",
        pwd: "*******",
        roles: [
            {
                role: "read",
                db: "NYC_Parking_Violations"
            }
        ]
    }
)
```

MapReduce & Aggregations

- O ***MapReduce*** é um paradigma de processamento de dados.

- ***Map*** – Realiza o mapeamento dos dados de entrada em pares chave-valor;

- ***Reduce*** – Realiza a agregação ou agrupamento de dados com base em chave-valor;



Aggregations Exemplo

- Vamos demonstrar as ***aggregations*** realizadas com recurso ao *software* ***VSCode***.

Visualização

É POSSÍVEL REALIZAR A VISUALIZAÇÃO DAS QUERIES NO POWER BI;

Outros Aspetos

APÓS MONTAR TODA A ARQUITETURA NO ***MONGODB***, AINDA FICARAM ALGUNS PONTOS POR APRIMORAR:

COMUNICAÇÃO DIRETA ENTRE ***POWER BI*** E ***MONOGDB***;

PRÉ-PROCESSAMENTO DOS DADOS DO ***DATASET***;

Conclusão

Podemos concluir que foi aplicado o modelo distribuído ***Replica Set*** em cada um dos 5 ***Shards***;

Foram ainda propostos 3 tipos de acesso o ***Admin***, ***Read-Write*** e o ***Read-Only*** com vários níveis de controlo de dados;

Finalmente o uso do ***MapReduce***, assim como da ***Aggregations*** demostrando a sua utilidade para extração de dados mais complexos;